



Final Project: IoT Deception Honeypot Network

Course: Offensive Technologies

Performed by: *Salekh Mamadaliev, Nikita Niakhai* — M25-SNE-01

Video Demo: https://drive.google.com/file/d/10S-Zc925qLTtUI_NBxL2D95CHJSIsQao/view?usp=drive_link

Abstract

This project presents a realistic IoT honeypot network designed to simulate smart devices such as IP cameras and MQTT sensors in order to attract real-world attackers and capture malicious activity. The system is deployed on a hardened VPS (Yandex Cloud, Ubuntu 24.04) using Docker and Docker Compose, running Cowrie (SSH/Telnet honeypot), a fake camera stub (nginx), and a Mosquitto MQTT broker as decoy services. All collected logs are forwarded to a Wazuh SIEM stack (manager, indexer, dashboard) for correlation, behavior classification (botnets, brute-force, MQTT flood), and real-time dashboard visualization. Fake device fingerprints and intentional weak credentials increase honeypot realism and attacker engagement. The project demonstrates end-to-end deception: from server hardening, through containerized decoy services, to custom Wazuh rules and validated attack simulations using Hydra, nmap, and mosquitto-clients.

Keywords: IoT, Honeypot, Cowrie, Wazuh, ELK, Docker, MQTT, Brute-force, Botnet, Attack Analytics, Deception

Table of Contents

- 0. Division of Labour
- 1. Introduction
- 2. Project Overview
 - 2.1 Problem Statement
 - 2.2 Technology Stack

3. Architecture Model

- 3.1 Infrastructure Setup
- 3.2 SSH Hardening

4. Server Security

- 4.1 UFW Firewall Rules
- 4.2 Wazuh Admin Password

5. Honeypot Layer

- 5.1 Cowrie — SSH/Telnet Honeypot
- 5.2 Fake Camera — nginx Stub
- 5.3 MQTT Broker — Mosquitto

6. Analytics — Wazuh SIEM

- 6.1 Wazuh Server Deployment
- 6.2 Wazuh Agent
- 6.3 Custom Rules for Cowrie
- 6.4 Dashboards and Alerts

7. Attack Simulation

- 7.1 SSH Brute-Force (Hydra → Cowrie)
- 7.2 Port Scanning (nmap)
- 7.3 MQTT Flood — Custom Script
- 7.4 Statistics

8. Future Development

9. Summary

10. References

Division of Labour

Nikita Niakhai	Salekh Mamadaliev
Setting up VPS in Yandex Cloud, initial access via ssh	User configuration on VPS, SSH hardening, UFW configuration

Wazuh Initial installation	Honeypot containers configuration
Wazuh hardening (credentials for all users inside a cluster)	Wazuh configuration for honeypot logs gathering
Attack simulation, script writting, validating attacks (brute force, reconisaince, mqtt flodd)	Wazuh custom rules creation for events generation and custom dashboard configuration
Diagram drawings, writting Report 1/2, Presentation review	Presentation creation, writing Report 1/2, Video editing

1. Introduction

The rapid growth of consumer and industrial IoT devices has produced a vast attack surface populated by under-secured embedded systems. Botnets such as Mirai and its descendants continue to weaponise default credentials and exposed management interfaces, especially on SSH/Telnet, HTTP camera panels, and MQTT brokers. This project deploys a **deceptive IoT environment** on a publicly reachable VPS to attract, observe, and classify such activity, while feeding all events into a Wazuh-based SIEM for correlation and visualization.

2. Project Overview

2.1 Problem Statement

The primary goal is to study attacker behavior targeting IoT devices by deploying believable decoy services on a publicly accessible VPS. Traditional security monitoring focuses on defending real assets; this project takes the opposite approach — deliberately exposing fake vulnerabilities to observe what attackers do when they believe they have found a real device. The project also serves as a practical demonstration of SIEM integration, log normalization, and real-time alerting for a non-traditional attack surface.

2.2 Technology Stack

Category	Technology	Purpose
SSH/Telnet Honeypot	Cowrie	Emulates SSH/Telnet services, captures credentials and commands
Fake Camera	nginx (stub)	Simulates an IP camera HTTP interface to attract scanners
MQTT Broker	Mosquitto	Decoy IoT message broker exposed to the internet

Category	Technology	Purpose
Log Shipping	Filebeat	Collects logs from containers and forwards to Wazuh
SIEM / Analytics	Wazuh + ELK Stack	Log correlation, alerting, dashboards, behavior classification
Containerization	Docker + Docker Compose	Orchestrates all honeypot and SIEM services
Infrastructure	VPS (Ubuntu 24.04)	Publicly accessible host for attracting real attack traffic

3. Architecture Model

The project runs on a **VPS** accessed via SSH with key-based authentication. All honeypot services and the Wazuh stack are deployed inside Docker containers, orchestrated with Docker Compose on the same host.

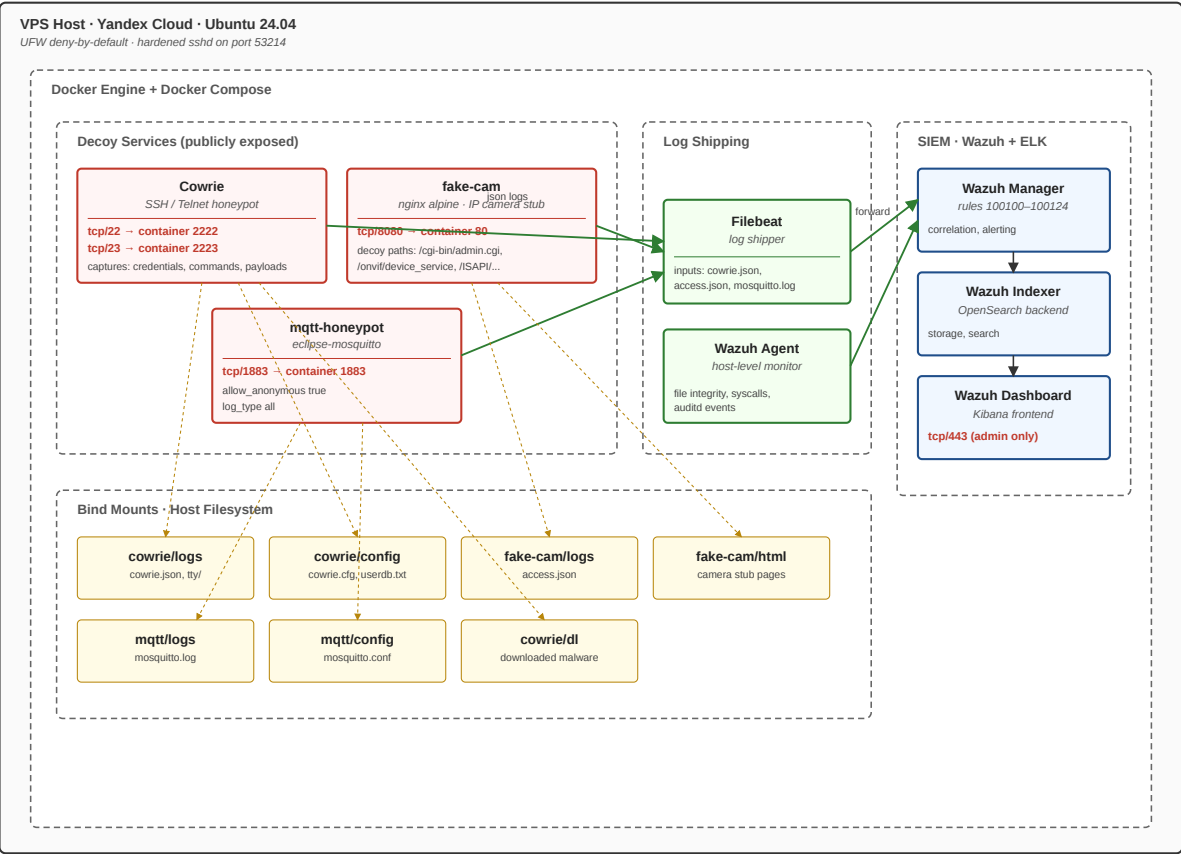


Figure. Components topology

The diagram shows three logical zones inside the Docker engine on a single hardened VPS: red-bordered decoy services (Cowrie, fake-cam, mqtt-honeypot), a green log-shipping layer (Filebeat plus host-level Wazuh Agent), and a blue Wazuh+ELK SIEM stack consuming those logs. Yellow dashed lines mark bind mounts to the host filesystem where configs, captured logs, and downloaded malware persist.

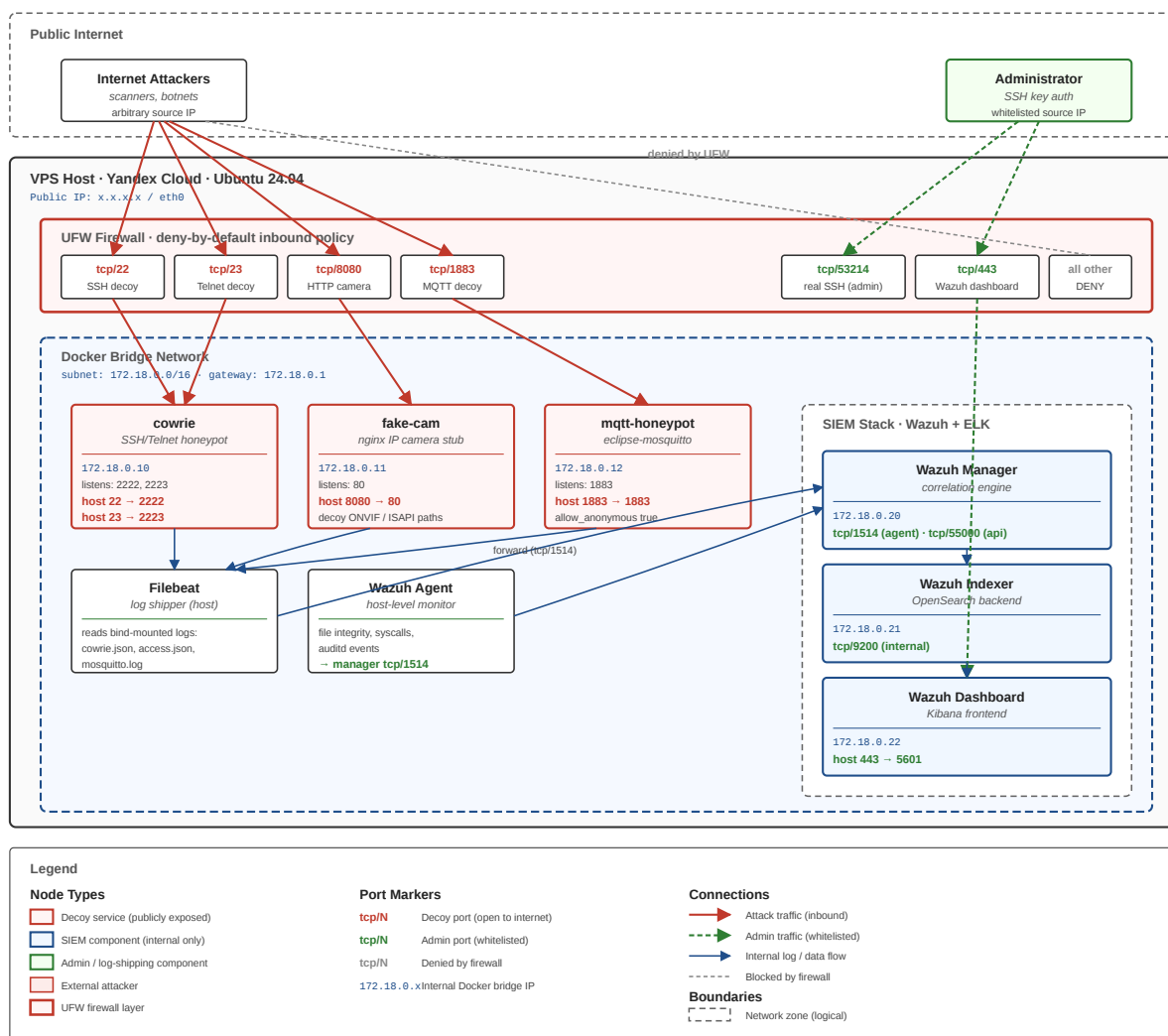


Figure. Network topology

The topology shows three trust zones stacked top-to-bottom: the public internet (attackers and the whitelisted administrator), the VPS host with UFW enforcing a deny-by-default policy that exposes only four decoy ports plus two admin ports, and the internal Docker bridge network (172.18.0.0/16) where decoy containers, log shippers, and the Wazuh+ELK stack communicate. Red arrows trace attack traffic into the decoys, green arrows mark the

segregated admin path to port 53214 and the dashboard, and blue arrows show the internal log pipeline feeding the SIEM.

3.1 Infrastructure Setup

- **VPS provider:** Yandex Cloud
- **OS:** Ubuntu 24.04
- **SSH hardened:** custom port 53214, PermitRootLogin disabled, PasswordAuthentication disabled, AllowUsers restricted
- **Docker + Docker Compose** installed
- **UFW** configured to expose only honeypot ports and Wazuh dashboard

3.2 SSH Hardening

The goal is to keep the real admin entry point invisible to scanners while leaving the decoy SSH (port 22) wide open.

- **Custom SSH configuration applied:**
 - Changed default port 22 → 53214
 - PermitRootLogin no
 - PasswordAuthentication no
 - AllowUsers restricted to project user only

Moves the real management plane off the default scanned port, disables the most-attacked authentication paths (root login, passwords), and restricts who can even attempt a session. The decoy SSH port 22 remains available, but it points at Cowrie — not the real shell.

```
root@vm-honeypot-net:/etc/ssh/sshd_config.d# cat 10-hardening.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
AllowUsers honey-eater
MaxStartups 3:50:10
```

Figure. sshd_config showing hardened SSH parameters on the VPS.

```

root@vm-honeypot-net:~/wazuh-docker/single-node# systemctl status ssh.socket
● ssh.socket - OpenBSD Secure Shell server socket
   Loaded: loaded (/usr/lib/systemd/system/ssh.socket; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/ssh.socket.d
            └─override.conf
   Active: active (running) since Thu 2026-04-30 06:26:06 UTC; 3h 46min ago
   Triggers: ● ssh.service
   Listen: 0.0.0.0:53214 (Stream)
            [::]:53214 (Stream)
   Tasks: 0 (limit: 4651)
   Memory: 12.0K (peak: 264.0K)
   CPU: 1ms
   CGroup: /system.slice/ssh.socket

Apr 30 06:26:06 vm-honeypot-net systemd[1]: Listening on ssh.socket - OpenBSD Secure Shell server socket

```

Figure. Verification of the SSH hardening (service status / login behavior).

4. Server Security

The server is hardened to ensure that only honeypot-related traffic reaches the decoy services, while the management plane (SSH) remains accessible only via key-based authentication.

4.1 UFW Firewall Rules

UFW is the boundary between the public internet and the running services.

- **Inbound policy:** UFW is configured to allow inbound traffic on honeypot ports (real SSH port `53214`, SSH honeypot on `22`, Telnet on `23`, HTTP camera stub on `8080`, MQTT on `1883`) and the Wazuh dashboard port, while blocking all other inbound traffic.

Explanation: The deny-by-default policy guarantees that only the deliberately exposed decoys and the management/SIEM ports are reachable. Each open port maps to a specific decoy service inside Docker, which keeps the attack surface predictable and easy to correlate in Wazuh.

```

root@vm-honeypot-net:~/wazuh-docker/single-node# ufw status
Status: active

To Action From
--
53214/tcp ALLOW Anywhere
22/tcp ALLOW Anywhere
23/tcp ALLOW Anywhere
1883/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
53214/tcp (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)
23/tcp (v6) ALLOW Anywhere (v6)
1883/tcp (v6) ALLOW Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)

```

Figure. Output of `ufw status` showing the active firewall rules on the VPS.

4.2 Wazuh Admin Password

The Wazuh default password for `admin` and `kibanaserver` must be replaced with a value that Filebeat can use without escaping issues.

- **Replace the default Wazuh admin password** with a simple alphanumeric value (no special characters) so Filebeat can authenticate without parsing problems.



For some reason default `kibanaserver` hash for default password was corrupted, so we regenerated and out logging with Filebeat started working.

- We followed this guide to change the credentials:

Changing the default password of Wazuh users - Deployment on Docker
User manual, installation and configuration guides. Learn how to get the most out of the Wazuh platform.

<https://documentation.wazuh.com/current/deployment-options/docker/changing-default-password.html#wazuh-indexer-user>



5. Honeypot Layer

Each service is a separate Docker container, so its blast radius is limited and its logs are isolated. All honeypot services run as isolated Docker containers. Each service simulates a different IoT device or protocol to maximize the attack surface presented to scanners and botnets.

```
root@vm-honeypot-net:/home/honey-eater/honeypot/compose# docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
cowrie              cowrie/cowrie:latest  "/cowrie/cowrie-env/..."  cowrie     2 weeks ago    Up 10 days
0.0.0.0:22->2222/tcp, [::]:22->2222/tcp, 0.0.0.0:23->2223/tcp, [::]:23->2223/tcp
fake-cam            nginx:alpine         "/docker-entrypoint..."  fake-cam   3 days ago     Up 3 days
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
mqtt-honeypot       eclipse-mosquitto:latest  "/docker-entrypoint..."  mqtt       3 days ago     Up 3 days
0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
root@vm-honeypot-net:/home/honey-eater/honeypot/compose#
```

Figure. `docker compose ps` showing the running honeypot containers.

All three decoy containers — `cowrie`, `fake-cam`, and `mqtt-honeypot` — are in the running state with their host ports correctly mapped (`22:2222`, `23:2223`, `8080:80`, `1883:1883`). This matches the compose definition below and confirms the honeypot layer is live and reachable.

```
services:
  cowrie:
    image: cowrie/cowrie:latest
    container_name: cowrie
    restart: unless-stopped
    ports:
      - "22:2222/tcp"
      - "23:2223/tcp"
    volumes:
      - ../cowrie/config/cowrie.cfg:/cowrie/cowrie-git/etc/cowrie.cf
g:ro
      - ../cowrie/config/userdb.txt:/cowrie/cowrie-git/etc/userdb.tx
t:ro
      - ../cowrie/logs:/cowrie/cowrie-git/var/log/cowrie
      - ../cowrie/dl:/cowrie/cowrie-git/var/lib/cowrie/dl
    environment:
      - COWRIE_TELNET_ENABLED=yes

  fake-cam:
    image: nginx:alpine
```

```

container_name: fake-cam
restart: unless-stopped
ports:
  - "8080:80"
volumes:
  - ../fake-cam/html:/usr/share/nginx/html:ro
  - ../fake-cam/logs:/var/log/nginx
  - ../fake-cam/nginx.conf:/etc/nginx/conf.d/default.conf:ro

mqtt:
  image: eclipse-mosquitto:latest
  container_name: mqtt-honeypot
  restart: unless-stopped
  ports:
    - "1883:1883"
  volumes:
    - ../mqtt/config:/mosquitto/config:ro
    - ../mqtt/logs:/mosquitto/log

```

5.1 Cowrie — SSH/Telnet Honeypot

Cowrie pretends to be a fully working shell, which lets it capture the entire post-login behaviour of an attacker.

- **Deploy Cowrie on default ports**
 - Listens on default ports **22** (SSH) and **23** (Telnet)
 - Fake device fingerprint configured to mimic a common router/camera
 - Logs stored in JSON format, collected by Filebeat

```

[honeypot]
hostname = iotcam-01
#log_path = /cowrie/log
#download_path = /cowrie/dl

[output_jsonlog]
enabled = true

```

```
[output_textlog]
enabled = true
```

cowrie.cfg

- **Seed the credential database with weak IoT defaults**

```
root:0:root
root:0:admin
root:0:123456
root:0:password
admin:0:admin
admin:0:1234
guest:0:guest
ubnt:0:ubnt
pi:0:raspberry
```

userdb.txt

Cowrie's `userdb.txt` defines which credentials are accepted. Including the canonical IoT/router default pairs (`root/root` , `admin/admin` , `ubnt/ubnt` , `pi/raspberry` , etc.) ensures that Mirai-style botnets, which iterate exactly these combinations, will succeed and continue into a fake shell — exposing their post-login payloads.

5.2 Fake Camera — nginx Stub

HTTP decoy mimics an IP camera login page and answers the most-scanned vulnerable paths to keep scanners interested.

- **Deploy the nginx stub on port 8080**
 - Returns a fake camera login page
 - HTTP access logs forwarded to Wazuh via Filebeat
 - Specific decoy endpoints (`/cgi-bin/admin.cgi` , `/onvif/device_service` , `/ISAPI/Security/userCheck`) respond instead of 404'ing

```
log_format json_combined escape=json
'{"time": "$time_iso8601", '
' "src_ip": "$remote_addr", '
' "method": "$request_method", '
' "uri": "$request_uri", '
' "status": $status, '
' "user_agent": "$http_user_agent"}';
```

```

access_log /var/log/nginx/access.json json_combined;

server {
    listen 80;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /cgi-bin/admin.cgi { return 200 ""; }
    location /onvif/device_service { return 200 ""; }
    location /ISAPI/Security/userCheck { return 401 ""; }
}

```

nginx.conf

- **Serve a believable IP camera login page**

```

<!DOCTYPE html>
<html>
<head><title>IPCamera Login</title></head>
<body style="background:#1a1a1a;display:flex;align-items:center;justify-content:center;height:100vh;margin">
    <div style="background:#2a2a2a;padding:40px;border-radius:4px;width:320px">
        <h2 style="color:#fff;text-align:center;margin:0 0 24px">IP Camera</h2>
        <p style="color:#888;text-align:center;font-size:12px">DS-2CD2143G2-I</p>
        <form method="POST" action="/login">
            <input name="username" placeholder="Username" value="admin" style="width:100%;padding:8px;margin:8px 0;box-sizing:border-box"><br>
            <input name="password" type="password" placeholder="Password" style="width:100%;padding:8px;margin:8px 0;box-sizing:border-box"><br>
            <button type="submit"

```

```
style="width:100%;padding:10px;background:#0066cc;color:#fff;
border:none;margin-top:8px;cursor:>
  Login
</button>
</form>
</div>
</body>
</html>
```

index.html

5.3 MQTT Broker — Mosquitto

A Mosquitto MQTT broker is exposed on port **1883** without authentication, simulating an insecure IoT message bus. This attracts MQTT-targeting tools and botnets that scan for open brokers.

- **Run Mosquitto with intentional weak configuration**
 - Listens on port **1883**
 - Anonymous connections allowed
 - Connection and publish logs forwarded to Wazuh

```
listener 1883
allow_anonymous true
log_dest file /mosquitto/log/mosquitto.log
log_type all
```

mosquitto.conf

allow_anonymous true is the misconfiguration that real-world IoT deployments leak the most often, so this is the decoy condition we want. **log_type all** ensures every connect/subscribe/publish/disconnect event lands in the file Filebeat is tailing.

6. Analytics — Wazuh SIEM

Without a SIEM, the decoys would just be noisy log files; Wazuh turns them into correlated alerts and dashboards.

Wazuh is deployed via Docker Compose and provides the full ELK-based SIEM stack: indexer, manager, and dashboard. All honeypot events flow into Wazuh for correlation, classification, and visualization.

6.1 Wazuh Server Deployment

All three Wazuh components share the same host as the honeypots.

- **Deploy Wazuh via the official Docker Compose configuration.** The Wazuh manager, indexer (OpenSearch), and dashboard run as separate containers on the same host.

Provides a self-contained SIEM stack on the same VPS without needing a second machine. The manager handles rules and decoders, the indexer stores events, and the dashboard is the analyst-facing UI.

```
root@vm-honeypot-net:~/wazuh-docker/single-node# docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE
single-node-wazuh.dashboard-1	wazuh/wazuh-dashboard:4.14.4	"/entrypoint.sh"	wazuh.dashboard
2 days ago	Up 2 days	443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp	
single-node-wazuh.indexer-1	wazuh/wazuh-indexer:4.14.4	"/entrypoint.sh open..."	wazuh.indexer
2 days ago	Up 2 days	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp	
single-node-wazuh.manager-1	wazuh/wazuh-manager:4.14.4	"/init"	wazuh.manager
2 days ago	Up 2 days	0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp	

```
root@vm-honeypot-net:~/wazuh-docker/single-node#
```

Figure. Wazuh services statuses

The dashboard is reachable and reports a healthy cluster — manager, indexer, and dashboard components are up. This confirms the Compose deployment succeeded and the SIEM is ready to ingest the honeypot logs described in Section 5.

6.2 Wazuh Agent

The agent watches the VPS itself, not the decoy containers.

- **Install the Wazuh agent directly on the VPS host** to monitor system-level events (file integrity, host-based intrusion detection) in addition to the honeypot log streams.

```

root@vm-honeypot-net:~/wazuh-docker/single-node# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-04-27 11:52:19 UTC; 2 days ago
     Tasks: 32 (limit: 4651)
   Memory: 533.2M (peak: 951.9M swap: 9.8M swap peak: 23.5M)
      CPU: 4min 47.675s
   CGroup: /system.slice/wazuh-agent.service
           └─3379452 /var/ossec/bin/wazuh-execd
             └─3379463 /var/ossec/bin/wazuh-agentd
               └─3379476 /var/ossec/bin/wazuh-syscheckd
                 └─3379487 /var/ossec/bin/wazuh-logcollector
                   └─3379503 /var/ossec/bin/wazuh-modulesd

Apr 27 11:52:13 vm-honeypot-net systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Apr 27 11:52:13 vm-honeypot-net env[3379426]: Starting Wazuh v4.14.4...
Apr 27 11:52:14 vm-honeypot-net env[3379426]: Started wazuh-execd...
Apr 27 11:52:15 vm-honeypot-net env[3379426]: Started wazuh-agentd...
Apr 27 11:52:15 vm-honeypot-net env[3379426]: Started wazuh-syscheckd...
Apr 27 11:52:16 vm-honeypot-net env[3379426]: Started wazuh-logcollector...
Apr 27 11:52:17 vm-honeypot-net env[3379426]: Started wazuh-modulesd...
Apr 27 11:52:19 vm-honeypot-net env[3379426]: Completed.
Apr 27 11:52:19 vm-honeypot-net systemd[1]: Started wazuh-agent.service - Wazuh agent.

```

Figure. Wazuh agent running

6.3 Custom Rules for Cowrie

- **Add custom rules to the Wazuh manager** to parse honeypot logs and generate alerts (100100 – 100124).

```

<group name="honeypot,">

  <!-- Cowrie SSH/Telnet -->
  <rule id="100100" level="5">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.session.connect</field>
    <description>Cowrie: New connection</description>
  </rule>

  <rule id="100101" level="8">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.login.failed</field>
    <description>Cowrie: Failed login attempt</description>
  </rule>

  <rule id="100102" level="12" frequency="5" timeframe="60">
    <if_matched_sid>100101</if_matched_sid>

```

```

    <same_field>data.src_ip</same_field>
    <description>Cowrie: Brute-force attack detected</description>
    <group>honeypot,brute_force,</group>
</rule>

<rule id="100103" level="10">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.login.success</field>
    <description>Cowrie: Successful honeypot login</description>
</rule>

<rule id="100104" level="12">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.command.input</field>
    <description>Cowrie: Command executed in honeypot</description>
</rule>

<rule id="100105" level="12">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.session.file_download</field>
    <description>Cowrie: Malware download attempt</description>
    <group>honeypot,malware,</group>
</rule>

<rule id="100106" level="6">
    <decoded_as>json</decoded_as>
    <field name="eventid">cowrie.client.fingerprint</field>
    <description>Cowrie: SSH client fingerprint recorded</description>
>
    <group>honeypot,recon,</group>
</rule>

<!-- Fake camera nginx -->
<rule id="100110" level="3">
    <decoded_as>json</decoded_as>
    <field name="uri">\.+</field>
    <description>Fake-cam: HTTP request</description>
</rule>

<rule id="100111" level="8">

```

```

    <if_sid>100110</if_sid>
    <field name="uri">cgi-bin|admin|ISAPI|onvif</field>
    <description>Fake-cam: IoT exploit path scan</description>
    <group>honeypot,recon,</group>
</rule>

<rule id="100112" level="10">
    <if_sid>100110</if_sid>
    <field name="uri">\.env|\.git|wp-admin|phpmyadmin|shell|cmd|exec
</field>
    <description>Fake-cam: Generic web exploit attempt</description>
    <group>honeypot,web_attack,</group>
</rule>

<!-- MQTT -->
<rule id="100120" level="5">
    <match>New connection from</match>
    <description>MQTT: New connection to honeypot broker</description>
>
    <group>honeypot,</group>
</rule>

<rule id="100121" level="6">
    <match>New client connected from</match>
    <description>MQTT: Client connected to honeypot broker</descripti
on>
    <group>honeypot,</group>
</rule>

<rule id="100122" level="10">
    <match>Received SUBSCRIBE</match>
    <description>MQTT: Wildcard subscription - possible topic enumera
tion</description>
    <group>honeypot,recon,</group>
</rule>

<rule id="100123" level="8">
    <match>Protocol error</match>
    <description>MQTT: Protocol error - possible scanner or exploit a
ttempt</description>

```

```

    <group>honeypot,scan,</group>
  </rule>

  <rule id="100124" level="10" frequency="10" timeframe="30">
    <if_matched_sid>100120</if_matched_sid>
    <description>MQTT: Multiple rapid connections - possible flood at
tack</description>
    <group>honeypot,flood,</group>
  </rule>

</group>

```

Defines a layered detection model: low-severity baseline events (100100 , 100110 , 100120), medium-severity recon detections (100106 , 100111 , 100122), and high-severity correlated alerts (100102 brute-force, 100124 MQTT flood) that fire only when a threshold is crossed inside a time window. The `same_field` and `if_matched_sid` directives are what turn isolated log lines into actual attack-pattern alerts.

6.4 Dashboards and Alerts

Simple yet informative dashboard was created in Wazuh web interface with charts and recent activity.

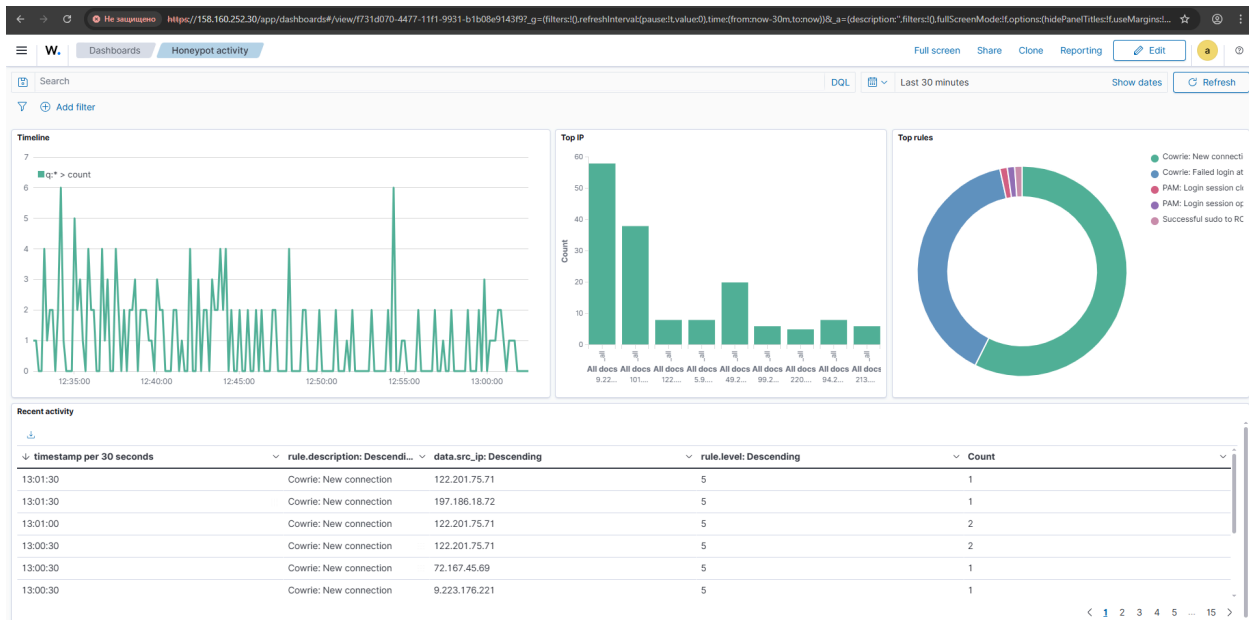


Figure. Wazuh custom dashboard with charts and a recent-events table for the honeypot rule group.

The dashboard shows live alert volume, rule-level distribution, and a recent-events table populated with honeypot rule IDs from the `100100` – `100124` range. This validates that the custom rules from Section 6.3 are being matched against real ingested logs and that the visualization layer is wired up correctly.

7. Attack Simulation

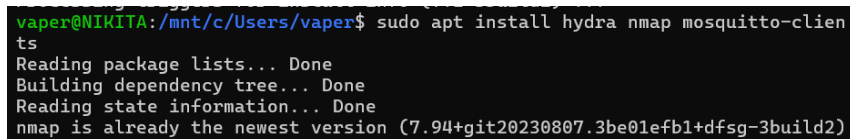
To validate the honeypot and SIEM pipeline, attack scenarios are executed from a separate machine (not the VPS) to generate realistic traffic that appears in the dashboards.

- **Install the attack tooling on the test client**

```
# Installing
sudo apt update && sudo apt upgrade -y
sudo apt install -y hydra nmap mosquitto-clients

# Installing community wordlists for hydra (optionally)
wget https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt
ls /usr/share/wordlists/
```

Installs the three tools used across this section: `hydra` for credential brute-force, `nmap` for port and service discovery, and `mosquitto-clients` for MQTT publish/subscribe. `rockyou.txt` is fetched as a realistic large password list for the brute-force runs.



```
vaper@NIKITA:/mnt/c/Users/vaper$ sudo apt install hydra nmap mosquitto-clients
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nmap is already the newest version (7.94+git20230807.3be01efb1+dfsg-3build2)
```

Figure. Terminal output of the attacker-side tooling installation.

The output confirms `hydra`, `nmap`, and `mosquitto-clients` were installed successfully on the test client and that `rockyou.txt` is available locally. The attack machine is now ready to execute the scenarios in 7.1–7.3.

7.1 SSH Brute-Force (Hydra → Cowrie)

The aim is to confirm that failed logins, successful logins, and the brute-force correlation rule (`100102`) all fire correctly.

- **Build custom user/password lists and launch Hydra against SSH and HTTP**

```
# Creating custom wordlist (passwords)
cat > ~/passwords.txt <<EOF
123456
password
admin
1234
root
123
qwerty
letmein
raspberry
admin
ubnt
guest
admin123
jbeD9e62Bs00
EOF

# Creating custom wordlist (users)
cat > ~/users.txt <<EOF
123456
password
admin
1234
root
123
qwerty
letmein
raspberry
ubnt
guest
admin123
kibanaserver
admin
EOF

# Brute-force SSH honeypot with common IoT credentials
hydra -L users.txt -P passwords.txt -s 22 -t 4 -V ssh://<VPS_IP>
```

```
# For form-based authentication
hydra -L users.txt -P passwords.txt -s 8080 -t 4 -V <VPS_IP> http-get /
```

Produces two small wordlists biased toward IoT defaults plus one known-good Cowrie credential (`jbeD9e62Bs00`) so we can verify successful-login alerts as well as failures. The first `hydra` command brute-forces SSH on port `22` (Cowrie); the second runs against the fake-cam HTTP on `8080` .

- manual passwords and users files + rockyou.txt downloaded

```
2026-05-01 13:06:12 (28.8 MB/s) - 'rockyou.txt' saved [139921497/139921497]
vaper@NIKITA:~$ ls
passwords.txt  rockyou.txt  users.txt
vaper@NIKITA:~$ cat passwords.txt users.txt
123456
password
admin
1234
root
123
qwerty
letmein
raspberry
admin
ubnt
guest
admin123
jbeD9e62Bs00
123456
password
admin
1234
root
123
qwerty
letmein
raspberry
ubnt
guest
admin123
kibanaserver
admin
```

Figure. Listing of the prepared wordlist files on the attacker host.

Both `users.txt` and `passwords.txt` are present with the expected contents, and `rockyou.txt` is in place as a fallback. The wordlist preparation step is complete, so the next Hydra run will draw from a representative IoT-default credential set.

- hydra runtime:

```
vaper@NIKITA:~$ hydra -L users.txt -P passwords.txt -o /home/vaper/hydra_found_passwords.txt -s 22 -t 4 -V ssh://158.160.252.30
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-01 13:13:13
[DATA] max 4 tasks per 1 server, overall 4 tasks, 196 login tries (l:14/p:14), ~49 tries per task
[DATA] attacking ssh://158.160.252.30:22/
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "123456" - 1 of 196 [child 0] (0/0)
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "password" - 2 of 196 [child 1] (0/0)

[ATTEMPT] target 158.160.252.30 - login "admin" - pass "admin" - 31 of 196 [child 2] (0/0)
[ATTEMPT] target 158.160.252.30 - login "admin" - pass "1234" - 32 of 196 [child 3] (0/0)
[22][ssh] host: 158.160.252.30 login: admin password: admin
[ATTEMPT] target 158.160.252.30 - login "1234" - pass "123456" - 43 of 196 [child 2] (0/0)
[22][ssh] host: 158.160.252.30 login: admin password: 1234
[ATTEMPT] target 158.160.252.30 - login "1234" - pass "password" - 44 of 196 [child 3] (0/0)

[ATTEMPT] target 158.160.252.30 - login "kibanaserver" - pass "jbeD9e62Bs00" - 182 of 196 [child 1] (0/0)
1 of 1 target successfully completed, 7 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-01 13:08:16
vaper@NIKITA:~$
```

Figure. Hydra in progress against the SSH honeypot — verbose login/password attempts streaming.

- Found credentials combinations:

```
vaper@NIKITA:~$ cat hydra_found_passwords.txt
# Hydra v9.5 run at 2026-05-01 13:13:13 on 158.160.252.30 ssh (hydra -L users.txt -P passwords.txt -o /home/vaper/hydra_found_passwords.txt -s 22 -t 4 -V ssh://158.160.252.30)
[22][ssh] host: 158.160.252.30 login: admin password: admin
[22][ssh] host: 158.160.252.30 login: admin password: 1234
[22][ssh] host: 158.160.252.30 login: root password: 123456
[22][ssh] host: 158.160.252.30 login: root password: password
[22][ssh] host: 158.160.252.30 login: root password: admin
[22][ssh] host: 158.160.252.30 login: ubnt password: ubnt
[22][ssh] host: 158.160.252.30 login: guest password: guest
```

Figure. Hydra final report showing the credential pairs that were accepted by the SSH honeypot.

Hydra reports successful `login:password` combinations that match entries seeded into Cowrie's `userdb.txt`. This confirms two things at once: Cowrie's fake authentication accepts the planted IoT defaults, and rule `100103` (Cowrie: Successful honeypot login) will have fired in Wazuh.

hydra command explained:

- `-L`: stands for logins, pointing to a file (`-l` for a single login string on the CLI)
- `-s`: stands for port
- `-V`: verbose mode / show login+pass combination for each attempt
- `-o`: output file
- Wazuh dashboard activity during brute force attack

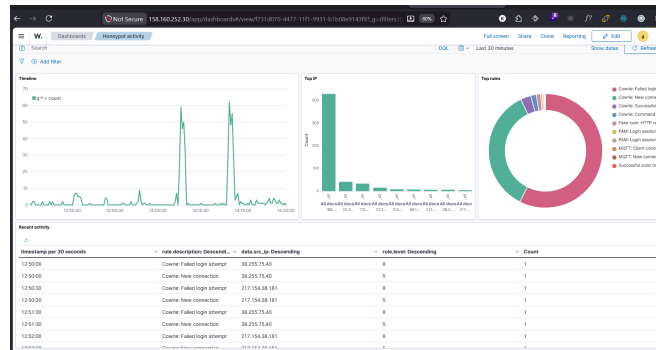


Figure. Wazuh dashboard during the Hydra brute-force run — alert volume spike and rule distribution.

- Log of successful sudo login

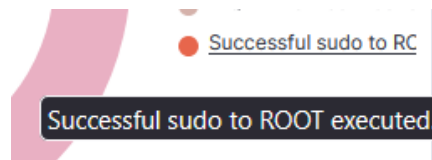


Figure. Wazuh alert for a successful sudo / privileged login event captured by the host agent.

This event comes from the Wazuh agent on the VPS host (Section 6.2), not from Cowrie. It demonstrates that host-level auditing works in parallel with the honeypot — if anyone ever genuinely escalates on the real host, it surfaces immediately, separate from the decoy traffic.

- Testing connection to the service of IOT camera device on port 22.

```
vaper@NIKITA:~$ ssh -o PreferredAuthentications=password admin@158.160.252.30
The authenticity of host '158.160.252.30 (158.160.252.30)' can't be established.
ED25519 key fingerprint is SHA256:ZyJ6dz3p9Suu3KyvuL0hj/yj7srNzxetiHjap1wkm90.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '158.160.252.30' (ED25519) to the list of known hosts.
admin@158.160.252.30's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
admin@iotcam-01:~$ whoami
admin
admin@iotcam-01:~$ ip a
-bash: /bin/ip: cannot execute binary file: Exec format error
admin@iotcam-01:~$ pwd
/home/admin
```

Figure. Manual SSH connection attempt to port 22 showing the Cowrie banner.

- Service on port 8080

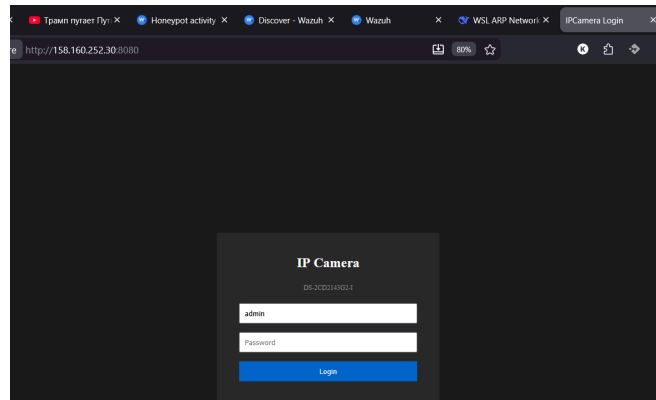


Figure. Browser view of the fake-cam login page served on port 8080.

The IP camera login UI from Section 5.2 renders as expected, including the model identifier `DS-2CD2143G2-I`. Any HTTP-aware scanner fingerprinting by page content will classify the host as a Hikvision-family camera, increasing the chance of a follow-up brute-force.

- Runtime: brute force on http server

```
honey@kali:~$ cat h_80_res.txt
Hydra v9.5 run at 2026-05-01 14:17:39 on 158.160.252.30 http-get (hydra) -L users.txt -P passwords.txt -o h_80_res.txt -s 8080 -t 4 -V 158
158.160.252.30 http-get /
[0000][http-get] host: 158.160.252.30 login: 123456 password: 123456
[0000][http-get] host: 158.160.252.30 login: 123456 password: password
[0000][http-get] host: 158.160.252.30 login: 123456 password: admin
[0000][http-get] host: 158.160.252.30 login: 123456 password: 1234
[0000][http-get] host: 158.160.252.30 login: password password: 123456
[0000][http-get] host: 158.160.252.30 login: password password: 1234
[0000][http-get] host: 158.160.252.30 login: admin password: 123456
[0000][http-get] host: 158.160.252.30 login: admin password: admin
[0000][http-get] host: 158.160.252.30 login: admin password: password
[0000][http-get] host: 158.160.252.30 login: admin password: 1234
[0000][http-get] host: 158.160.252.30 login: 1234 password: password
[0000][http-get] host: 158.160.252.30 login: 1234 password: 123456
[0000][http-get] host: 158.160.252.30 login: 1234 password: 1234
[0000][http-get] host: 158.160.252.30 login: 1234 password: admin
[0000][http-get] host: 158.160.252.30 login: root password: 123456
[0000][http-get] host: 158.160.252.30 login: root password: admin
[0000][http-get] host: 158.160.252.30 login: root password: 1234
[0000][http-get] host: 158.160.252.30 login: 123 password: 123456
[0000][http-get] host: 158.160.252.30 login: 123 password: password
```

Figure. Hydra in progress against the HTTP `/login` endpoint of the fake camera.

- Results: brute force on http server

```
honey@kali:~$ hydra -L users.txt -P passwords.txt -o h_80_res.txt
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal
uses (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-01 14:17:39
[DATA] max 4 tasks per 1 server, overall 4 tasks, 196 login tries (L:10/P:14), ~49 tries per task
[DATA] attacking http-get://158.160.252.30:8080/
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "123456" - 1 of 196 [child 0] (0/0)
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "password" - 2 of 196 [child 1] (0/0)
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "admin" - 3 of 196 [child 2] (0/0)
[ATTEMPT] target 158.160.252.30 - login "123456" - pass "1234" - 4 of 196 [child 3] (0/0)
[Done] [http-get] host: 158.160.252.30 login: 123456 password: 123456
[ATTEMPT] target 158.160.252.30 - login "password" - pass "123456" - 15 of 196 [child 0] (0/0)
```

Figure. Hydra final summary for the HTTP brute-force run.

7.2 Port Scanning (nmap)

- Run a targeted version scan and a deeper OS-detection scan

```
# Scan exposed honeypot ports
```

```
nmap -sV -p 22,23,80,1883,8080,443,2222,2323,1883,8080 <VPS_IP>
```

```
# Full scan with OS detection
nmap -sS -sV -O -p 1-10000 -T4 <VPS_IP>
```

The first command does a focused service-version scan on the ports we expect to be open (the honeypots). The second is a much louder SYN + version + OS scan over the first 10 000 ports — the kind a Shodan-style scanner might run — which lets us verify that nothing leaks beyond the intended honeypot ports.

- nmap known ports, results:

```
vaper@NIKITA:~$ nmap -sV -T4 -p 22,23,80,1883,8080,443,2222,2323,1883,8080 158.160.252.30
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-01 13:25 +03
WARNING: Duplicate port number(s) specified. Are you alert enough to be using Nmap? Have
some coffee or Jolt(tm).
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 60.00% done; ETC: 13:25 (0:00:11 remaining)
Stats: 0:02:03 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 80.00% done; ETC: 13:27 (0:00:31 remaining)
Nmap scan report for 158.160.252.30
Host is up (0.028s latency).

PORT      STATE SERVICE          VERSION
22/tcp    open  ssh              OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
23/tcp    open  telnet?
80/tcp    filtered http
443/tcp   open  ssl/https
1883/tcp   open  mosquitto version 2.1.2
2222/tcp  filtered EtherNetIP-1
2323/tcp  filtered 3d-nfsd
8080/tcp   open  http              nginx 1.29.8
```

Figure. nmap version-scan results for the targeted honeypot port set.

Only the deliberately exposed honeypot ports are open (22 , 23 , 1883 , 8080); everything else is filtered. The reported services match what we expect: Cowrie on 22/23 , Mosquitto on 1883 , nginx on 8080 . The real management SSH on 53214 does not appear in this scan range, confirming the firewall/hardening from Section 4.

- nmap Fingerprint:

```

=====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
SF-Port23-TCP:V=7.94SVN%I=7%D=5/1%Time=69F47F9E%P=x86_64-pc-linux-gnu%r(NU
SF:LL,7,"Login:\x20")%r(GenericLines,2C,"login:\x20\xff\xfb\x01Password:\x
SF:20\nLogin\x20incorrect\nlogin:\x20")%r(tn3270,16,"login:\x20\xff\xfe\x1
SF:8\xff\xfe\x19\xff\xfc\x19\xff\xfe\x0\xff\xfc\x0")%r(GetRequest,2C,"Login:
SF:\x20\xff\xfb\x01Password:\x20\nLogin\x20incorrect\nlogin:\x20")%r(HTTPO
SF:ptions,2C,"login:\x20\xff\xfb\x01Password:\x20\nLogin\x20incorrect\nlog
SF:in:\x20")%r(RTSPRequest,2C,"login:\x20\xff\xfb\x01Password:\x20\nLogin\
SF:x20incorrect\nlogin:\x20")%r(RPCCheck,7,"login:\x20")%r(DNSVersionBindR
SF:eqTCP,7,"login:\x20")%r(DNSStatusRequestTCP,7,"login:\x20")%r(Hello,14,"
SF:login:\x20\xff\xfb\x01Password:\x20")%r(SSLSessionReq,14,"login:\x20\x
SF:f\xfb\x01Password:\x20")%r(TerminalServerCookie,14,"login:\x20\xff\xfb\
SF:x01Password:\x20")%r(Kerberos,14,"login:\x20\xff\xfb\x01Password:\x20")
SF:%r(X11Probe,7,"login:\x20")%r(FourOhFourRequest,2C,"login:\x20\xff\xfb\
SF:x01Password:\x20\nLogin\x20incorrect\nlogin:\x20")%r(LPDString,14,"logi
SF:n:\x20\xff\xfb\x01Password:\x20")%r(LDAPSearchReq,2C,"login:\x20\xff\x
SF:b\x01Password:\x20\nLogin\x20incorrect\nlogin:\x20")%r(LDAPBindReq,7,"l
SF:login:\x20")%r(SIPOptions,BE,"login:\x20\xff\xfb\x01Password:\x20\nLogin
SF:\x20incorrect\nlogin:\x20Password:\x20\nLogin\x20incorrect\nlogin:\x20P
SF:assword:\x20\nLogin\x20incorrect\nlogin:\x20Password:\x20\nLogin\x20inc
SF:orrect\nlogin:\x20Password:\x20\nLogin\x20incorrect\nlogin:\x20Password
SF::\x20")%r(LANDesk-RC,7,"login:\x20")%r(TerminalServer,7,"login:\x20")%r
SF:(NotesRPC,7,"login:\x20")%r(JavaRMI,7,"login:\x20")%r(WMSRequest,7,"log
SF:in:\x20")%r(afp,7,"login:\x20")%r(giop,7,"login:\x20");
=====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
SF-Port443-TCP:V=7.94SVN%T=SSL%I=7%D=5/1%Time=69F47F9E%P=x86_64-pc-linux-g
SF:nu%r(GetRequest,157,"HTTP/1.1\x20302\x20Found\r\nlocation:\x20/app/log
SF:in/?r\nosd-name:\x20wazuh\ .dashboard\r\nx-frame-options:\x20sameorigin
SF:r\nocache-control:\x20private,\x20no-cache,\x20no-store,\x20must-revali
SF:date\r\nset-cookie:\x20security_authentication=\x20Max-Age=0;\x20Expir
SF:es=Thu,\x2001\x20Jan\x201970\x2000:00:00\x20GMT;\x20HttpOnly;\x20Path=/
SF:r\ncontent-length:\x200\r\nDate:\x20Fri,\x2001\x20May\x202026\x2010:25
SF:33\x20GMT\r\nConnection:\x20close\r\n\r\n")%r(HTTPOptions,146,"HTTP/1\
SF:1\x20404\x20Not\x20Found\r\nosd-name:\x20wazuh\ .dashboard\r\nx-frame-o
SF:ptions:\x20sameorigin\r\ncontent-type:\x20application/json;\x20charset=
SF:utf-8\r\nocache-control:\x20private,\x20no-cache,\x20no-store,\x20must-r
SF:validate\r\ncontent-length:\x2060\r\nDate:\x20Fri,\x2001\x20May\x20202
SF:6\x2010:25:34\x20GMT\r\nConnection:\x20close\r\n\r\n")%r(statusCode":404
SF:"error":\x20Not\x20Found","message":\x20Not\x20Found")%r(FourOhFou
SF:rRequest,1C4,"HTTP/1.1\x20401\x20Unauthorized\r\nosd-name:\x20wazuh\ .d
SF:ashboard\r\nx-frame-options:\x20sameorigin\r\ncontent-type:\x20applicat
SF:ion/json;\x20charset=utf-8\r\nocache-control:\x20private,\x20no-cache,\x
SF:20no-store,\x20must-revalidate\r\nset-cookie:\x20security_authentication
SF:n=\x20Max-Age=0;\x20Expires=Thu,\x2001\x20Jan\x201970\x2000:00:00\x20G
SF:MT;\x20HttpOnly;\x20Path=/\r\ncontent-length:\x2077\r\nDate:\x20Fri,\x2
SF:001\x20May\x202026\x2010:25:34\x20GMT\r\nConnection:\x20close\r\n\r\n")
SF:"statusCode":401,"error":\x20Unauthorized","message":\x20Authenticati
SF:on\x20required")%r(RTSPRequest,146,"HTTP/1.1\x20404\x20Not\x20Found
SF:r\nosd-name:\x20wazuh\ .dashboard\r\nx-frame-options:\x20sameorigin\r\n
SF:content-type:\x20application/json;\x20charset=utf-8\r\nocache-control:\x
SF:20private,\x20no-cache,\x20no-store,\x20must-revalidate\r\ncontent-leng
SF:th:\x2060\r\nDate:\x20Fri,\x2001\x20May\x202026\x2010:25:39\x20GMT\r\nC
SF:onnection:\x20close\r\n\r\n")%r(statusCode":404,"error":\x20Not\x20Found
SF:"","message":\x20Not\x20Found")%r(Hello,1C,"HTTP/1.1\x20400\x20Bad\x
SF:20Request\r\n\r\n")%r(SSLSessionReq,1C,"HTTP/1.1\x20400\x20Bad\x20Requ
SF:est\r\n\r\n");

```

Figure. nmap service/version fingerprints returned for the open ports.

The fingerprints returned by `-sV` look plausible to an automated scanner — Cowrie returns an OpenSSH-like banner consistent with a typical IoT device, the HTTP service identifies as nginx, and Mosquitto announces itself on `1883`. The honeypot-as-IoT illusion holds up against version detection.

- OS detected:

```

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

Figure. nmap `-o` OS-detection result for the VPS.

- nmap full scan results:

```
vaper@NIKIITA:~$ sudo nmap -sS -sV -O -p 1-10000 -T4 158.160.252.30
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-01 13:30 +03
Nmap scan report for 158.160.252.30
Host is up (0.026s latency).
Not shown: 9992 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
22/tcp    open  ssh              OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
23/tcp    open  telnet?
443/tcp   open  ssl/https
1514/tcp  open  fujitsu-dtcns?
1515/tcp  open  ssl/igor-protocol?
1883/tcp  open  mosquitto version 2.1.2
8080/tcp  open  http              nginx 1.29.8
9200/tcp  open  ssl/wap-wsp?
4 services unrecognized despite returning data. If you know the service/version, please submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?new-service :
=====NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)=====
SF-Port23-TCP:V=7.94SVN%I=7%D=5/1%Time=69F480DA%P=x86_64-pc-linux-gnu%r(NU
SF:LL,7,"login:\x20")%r(GenericLines,2C,"login:\x20\xff\xfb\x01Password:\x
SF:20\nLogin\x20incorrect\nlogin:\x20")%r(tn3270,16,"login:\x20\xff\xfe\x1
SF:8\xff\xfe\x19\xff\xfc\x19\xff\xfe\x0\xff\xfc\x0")%r(GetRequest,2C,"login:
SF:\x20\xff\xfb\x01Password:\x20\nLogin\x20incorrect\nlogin:\x20")%r(HTTP0
SF:ptions,2C,"login:\x20\xff\xfb\x01Password:\x20\nLogin\x20incorrect\nlog
SF:in:\x20")%r(RTSPRequest,2C,"login:\x20\xff\xfb\x01Password:\x20\nLogin\
SF:x20incorrect\nlogin:\x20")%r(RPCCheck,7,"login:\x20")%r(DNSVersionBindR
SF:eqTCP,7,"login:\x20")%r(DNSStatusRequestTCP,7,"login:\x20")%r(Help,14,"
SF:login:\x20\xff\xfb\x01Password:\x20")%r(SSLSessionReq,14,"login:\x20\x
SF:f\xfb\x01Password:\x20")%r(TerminalServerCookie,14,"login:\x20\xff\xfb\
SF:x01Password:\x20")%r(Kerberos,14,"login:\x20\xff\xfb\x01Password:\x20")
SF:%r(X11Probe,7,"login:\x20")%r(FourOhFourRequest,2C,"login:\x20\xff\xfb\
SF:x01Password:\x20\nLogin\x20incorrect\nlogin:\x20")%r(LPDString,14,"logi
```

Figure. nmap deep scan over ports 1–10000 with SYN, version, and OS detection.

7.3 MQTT Flood — Custom Script

The aim is to trigger both the recon rule (wildcard subscribe) and the flood-correlation rule (100124).

- Test the broker, then flood it, then enumerate topics

```
# OPTION 1
# Testing connection
mosquitto_sub -h <VPS_IP> -p 1883 -t "#" -v &

# Flooding with messages
for i in $(seq 1 200); do
    mosquitto_pub -h 158.160.252.30 -p 1883 -t "sensor/temp" -m "{\"v
al\":\"$i\"}"
done

# Recon, susbscribing on all topics
mosquitto_sub -h 158.160.252.30 -p 1883 -t "#" -v -C 50
```

The first command opens a wildcard subscription (#) to confirm the broker is reachable and to trigger the topic-enumeration rule (100122). The shell loop then publishes 200 messages in rapid succession, which simulates a small MQTT flood and exercises the rate-based rule

(100124). The third command does a bounded recon subscription that captures up to 50 messages.

- Connection tested:

```
vaper@NIKITA:~$ mosquitto_sub -h 158.160.252.30 -p 1883 -t "#" -v &  
[1] 4751
```

Figure. Successful `mosquitto_sub` connection to the honeypot broker.

The wildcard subscription connects without authentication, confirming `allow_anonymous true` from Section 5.3 is in effect. From the attacker's view, this is a recognisable misconfiguration signal that real-world MQTT botnets actively scan for.

- Flooding with messages:

```
vaper@NIKITA:~$ for i in $(seq 1 200); do mosquitto_pub -h 158.160.252.30 -p 1883 -t "sensor/temp" -m "{\"val\":$i}"; done  
done  
sensor/temp {"val":1}  
sensor/temp {"val":2}  
sensor/temp {"val":3}  
sensor/temp {"val":4}  
sensor/temp {"val":5}  
sensor/temp {"val":6}  
sensor/temp {"val":7}  
sensor/temp {"val":8}  
sensor/temp {"val":9}  
sensor/temp {"val":10}  
sensor/temp {"val":11}  
sensor/temp {"val":12}  
sensor/temp {"val":13}  
sensor/temp {"val":14}  
sensor/temp {"val":15}  
sensor/temp {"val":16}
```

Figure. Output of the `mosquitto_pub` flood loop publishing 200 messages.

The publish loop completes without errors, meaning the broker accepted all 200 messages on `sensor/temp`. This is the input that produces a burst of `New connection from` log lines on the broker side and feeds rule 100124 (`>=10` connections within 30s).

- Flood messages observed:

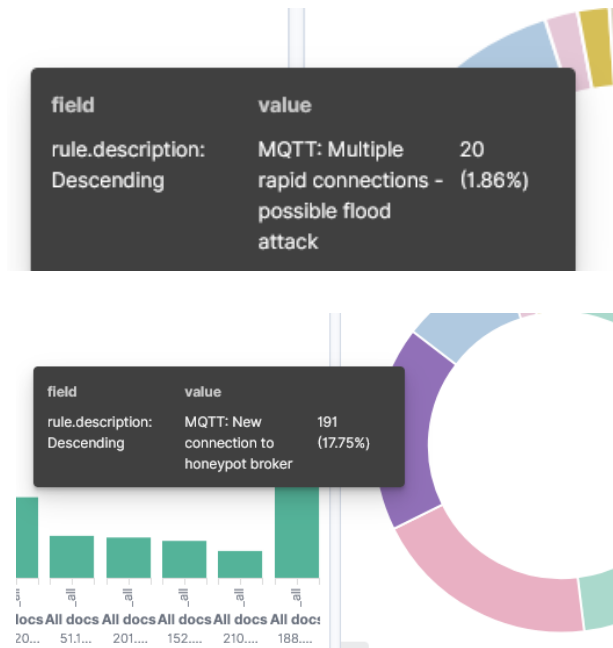
```
vaper@NIKITA:~$ mosquitto_sub -h 158.160.252.30 -p 1883 -t "test/#"  
-v  
test/hello ping  
test/hello ping  
sensor/temp {"val":1}  
sensor/temp {"val":2}  
sensor/temp {"val":3}  
sensor/temp {"val":4}  
sensor/temp {"val":5}  
sensor/temp {"val":6}  
sensor/temp {"val":7}  
sensor/temp {"val":8}  
sensor/temp {"val":9}
```

Figure. Subscriber terminal receiving the flooded messages in real time.

The wildcard subscriber receives the flood payloads (`{"val":1}` ... `{"val":200}`), proving the broker is correctly relaying messages. For the SIEM, every publish is a logged event — so the per-message volume here corresponds directly to rule-match volume in Wazuh.

MQTT subscription testing (first starting subscription 1, then sending messages 2, observing messages via subscription)

- MQTT logs about possible flood attack and new connections established



- MQTT subscription rules triggered

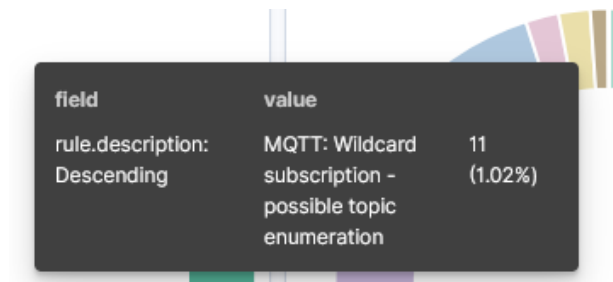


Figure. Wazuh alerts pane showing MQTT honeypot rules firing — including the wildcard-subscription recon rule.

7.4 Statistics

Since the honeypot machine is accessible on the Internet, different scanners, bots, and crawlers are actively trying to attack it. This gives us a good opportunity to gather some real information about scanners activity.

Created dashboard provides most crucial information about events: top IP addresses, rule description, and recent activity. In a real-life scenario, this potentially will help the SOC or monitoring team immediately react to possible threats.

We will observe small timeline from May 3rd 22:40 to May 5th 10:40 Moscow time.

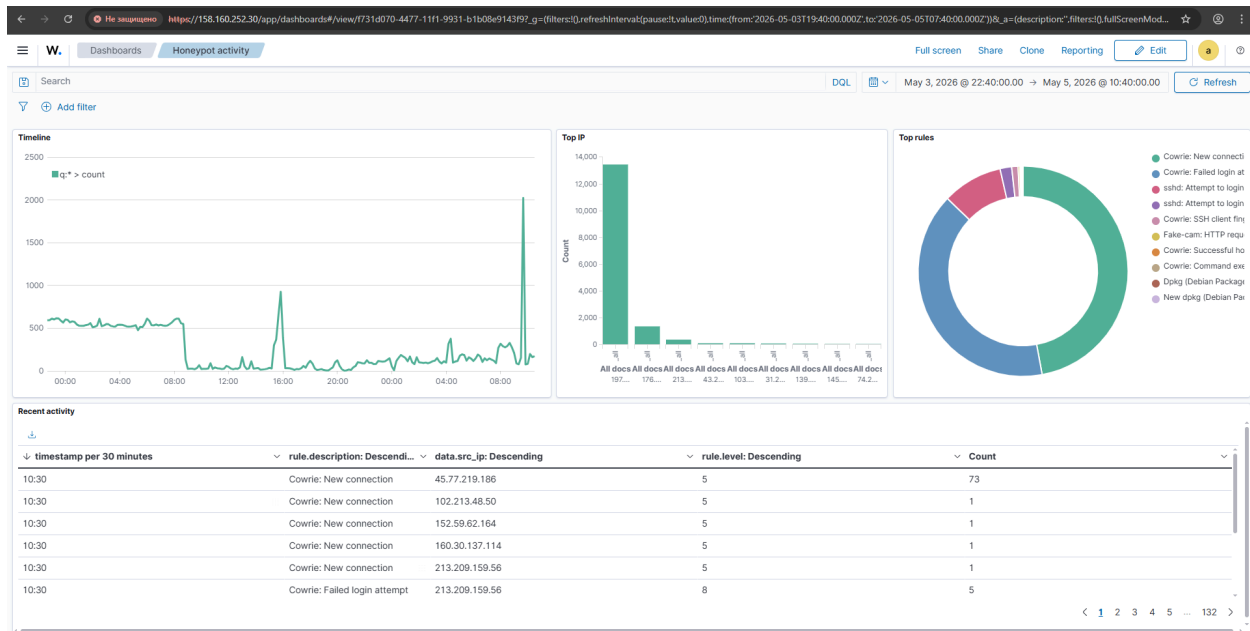


Figure. Timeline statistics

For chosen timeline we can see a total of **52,293 events**. Statistics for top rules by count:

Rule	Count
Cowrie: New connection	23,657
Cowrie: Failed login attempt	20,122
sshd: Attempt to login using a non-existent user	4,673
sshd: Attempt to login using a denied user.	897
Cowrie: SSH client fingerprint recorded	491

Also we can inspect top IP addresses by count:

IP	Count
197.242.182.66	13,456
176.65.139.95	1,380
213.171.199.40	391
43.241.37.250	120
103.179.216.97	120

From this, we can conclude that any machine that is accessible from the internet is constantly under attack from scanners, bots, and others. If interesting services or open

access are detected on the machine, attackers will try to actively attack the system in order to gain access to its computing power or sensitive data. In a real system, when attack patterns are detected, it is preferable to take some protective measures, such as blocking the attacking addresses.

8. Future Development

- ☐ Add **Dionaea** honeypot for FTP/SMB/Telnet protocol coverage
 - ☐ Implement **GeoIP enrichment** in Wazuh to map attacker origins
 - ☐ Add **automated threat intelligence lookup** (e.g. AbuseIPDB) for detected IPs
 - ☐ Extend MQTT decoy with **fake device topics and retained messages** for deeper engagement
 - ☐ Explore integration with **threat-sharing platforms** (MISP, OpenCTI)
-

9. Summary

The project successfully delivered a working IoT-themed deception network on a single hardened VPS. Three decoy services — Cowrie (SSH/Telnet), an nginx-based fake IP camera on port `8080`, and an open Mosquitto MQTT broker on `1883` — were deployed via Docker Compose and exposed to the public internet. A full Wazuh SIEM stack (manager, indexer, dashboard) was deployed on the same host, fed by Filebeat, and extended with a dedicated honeypot rule group (IDs `100100` – `100124`) covering connections, failed and successful logins, command execution, malware downloads, recon paths, web exploitation patterns, MQTT topic enumeration, and MQTT flood correlation. End-to-end validation was performed using Hydra, nmap, and `mosquitto-clients` from a separate machine, and every attack pattern produced the expected alerts and dashboard signals. SSH and firewall hardening (custom port `53214`, key-only auth, deny-by-default UFW) ensured that the only externally reachable surfaces were the intended decoys, while a Wazuh agent on the host provided independent host-level monitoring. The combination of believable device fingerprints (camera model name, IoT-default credentials, anonymous MQTT) and structured logging produces a platform that is realistic enough to attract opportunistic attackers and analytical enough to classify their behaviour into named patterns (brute-force, recon, flood) automatically.

10. References

- [Cowrie Documentation](#)

- [Wazuh Documentation](#)
- [Mosquitto MQTT Broker](#)
- [Filebeat Documentation](#)
- [Docker Compose Documentation](#)
- [Hydra — THC](#)
- [nmap Documentation](#)